OXFORD CRM – MASTER PROJECT DOCUMENTATION

> [!IMPORTANT]

> **HANDOVER DOCUMENT**

> This is the official project handover document for Oxford CRM. It contains everything necessary for a new developer or an AI assistant to understand, maintain, troubleshoot, and continue development of the CRM.

SECTION 1: PROJECT OVERVIEW

Project Name: Oxford CRM (WhatsApp API Integration)

Purpose: A centralized, WhatsApp-integrated Customer Relationship Management (CRM) system designed to track student inquiries, course admissions, staff performance, and revenue analytics for Oxford Computers.

Business Objectives:

- Eliminate manual lead tracking by automatically capturing WhatsApp interactions.

- Provide actionable intelligence on lead temperature (Hot/Warm/Cold).

- Drive accountability through strict staff assignment and event-driven attribution.

- Provide comprehensive analytics for management to optimize counseling efforts.

Intended Users:

- **Management (Admin)**: Full access to all dashboards, analytics, operations queues, and staff management.

- **Counselors (Staff)**: Responsible for following up with leads, changing statuses, and marking admissions.

Admin Responsibilities: Reviewing health metrics, managing staff accounts, monitoring overall conversion rates, and handling critical follow-ups.

Staff Responsibilities: Acting on hot queues, answering inquiries via the manual messaging interface, updating lead statuses, and logging course admissions.

Current Development Status: Phase 9.2A-Lite (Staff Master Data Management completed). Nearing full Role-Based Access Control (RBAC).

Major Features Completed: Lead Pipeline, WhatsApp Event Sourcing, Course Journey Tracking, Multi-Course Admissions, Action Center Priority Queues, CRM Health Dashboard, Staff Performance Analytics, Event-Driven Accountability, and JSON-based Staff Registry.

SECTION 2: TECHNOLOGY STACK

Backend Language: Python 3

Chosen for its rapid prototyping capabilities, extensive data processing libraries, and excellent integration with web frameworks and database ORMs.

Web Framework: Flask

Chosen for its lightweight footprint and modularity (Blueprints), allowing the CRM to scale iteratively without the heavy boilerplate of Django.

Database: PostgreSQL

Chosen for strict ACID compliance, JSONB support (vital for event payloads), and robust production scalability.

ORM: SQLAlchemy

Chosen to abstract SQL queries and manage complex analytical aggregations entirely in-memory using bulk querying, bypassing N+1 query problems.

Deployment Platform: Railway

Chosen for seamless CI/CD, auto-scaling, and managed PostgreSQL databases.

Frontend Integration: Jinja2 (Template Engine), Vanilla JavaScript, Bootstrap 5 (CSS)

Chosen to allow rapid UI development directly coupled with the backend routing, without the overhead of a separate SPA frontend.

WhatsApp Integration: Meta Official Cloud API

Chosen for verified business messaging, template support, and reliable webhook delivery.

Analytics Engine: In-memory Python Aggregation

Chosen to eliminate expensive `GROUP BY` database queries. By loading states via a single bulk query, the CRM maintains O(L+E) time complexity while avoiding database proxy disconnects.

SECTION 3: APPLICATION ARCHITECTURE

Flask Structure: The application uses the application factory pattern (`app/__init__.py`).

Blueprint Structure: The CRM is heavily modularized. `admin_bp` (in `app/routes/admin.py`) houses all internal CRM dashboards, analytics, and operational queues.

Route Organization: Webhook routes (receiving WA messages) are separated from internal CRM routes. CRM routes are protected via a query parameter `?key=ADMIN_KEY`.

Template Organization: Stored in `/templates`. Uses standard Jinja2 inheritance (though many dashboards are standalone for rapid iteration). Heavy use of `crm_style.css` for consistent, premium UI.

Static Assets: Stored in `/static/`. Contains `crm_style.css` and custom JS scripts.

Configuration Loading: Managed via `config.py` using `os.getenv`.

Environment Variables: Includes `DATABASE_URL`, `WHATSAPP_TOKEN`, `VERIFY_TOKEN`, `ADMIN_KEY`, `FLASK_SECRET_KEY`.

Database Initialization: Done via SQLAlchemy `db.create_all()` in the app factory context.

Request Lifecycle:

1. External WhatsApp webhooks hit the endpoint, updating `ConversationState` and appending to `LeadEvent`.

2. Staff accesses `/crm/operations?key=...`.

3. The server validates the key, runs a bulk query on `ConversationState` and `LeadEvent`.

4. In-memory Python loops aggregate data into actionable KPIs (Action Center queues, Admission metrics).

5. Jinja2 renders the HTML and delivers it to the browser.

SECTION 4: DATABASE DOCUMENTATION

> [!CAUTION]

> **Production Rule**: Avoid schema changes and migrations unless absolutely necessary to prevent breaking existing analytics or locking the database.

1. `ConversationState`

Purpose: Represents the current snapshot of a lead in the CRM pipeline.

Columns:

- `phone` (String, PK): The unique WhatsApp phone number.

- `name` (String): The user's WhatsApp profile name or CRM overridden name.

- `lead_status` (String): Current pipeline stage (e.g., 'Lead', 'Contacted', 'Enrolled').

- `lead_score` (Integer): The computed intelligence score (0-100).

- `assigned_staff` (String): Legacy text string linking the lead to a staff member.

- `is_admitted` (Boolean): Master admission flag.

- `course` (String): AI-detected course interest.

- `offer_course` (String): Staff-overridden course interest.

- `created_at` / `updated_at` (DateTime): Auditing timestamps.

Usage Notes: Do not use for historical tracking. This table only ever holds the *current* state.

2. `LeadEvent`

Purpose: An append-only event sourcing table that records every significant business action.

Columns:

- `id` (Integer, PK): Auto-increment.

- `phone` (String): Foreign key link to `ConversationState`.

- `event_type` (String): The exact action that occurred (e.g., `COURSE_ADMISSION`).

- `event_data` (Text/JSON): Extensible payload for the event (e.g., `{"course": "Python", "staff": "Kiran"}`).

- `created_at` (DateTime): Immutable timestamp.

Usage Notes: NEVER update or delete rows in this table. All analytics read this table to reconstruct timelines and attribution.

3. `ConversationMessage` (Message Log)

Purpose: Represents raw chat transcripts (WhatsApp messages).

Columns:

- `id` (Integer, PK).

- `phone` (String): Link to lead.

- `direction` (String): 'incoming' or 'outgoing'.

- `message` (Text): Message body.

- `source` (String): 'bot', 'manual', 'user'.

- `staff_name` (String): Populated for manual replies.

SECTION 5: EVENT SYSTEM DOCUMENTATION

The Oxford CRM relies on an Event-Driven Architecture. All historical data and score points are calculated from these events.

- **LEAD_CREATED**: Fired when a new number interacts. Payload: `{}`. Significance: Tracks raw lead acquisition.

- **COURSE_VIEWED**: Fired when AI/Bot detects a course query. Payload: `{"course": "Name"}`. Impact: +5 lead score.

- **COURSE_ENQUIRY**: Formalized interest. Payload: `{"course": "Name"}`. Impact: +10 lead score. Used in Course Journey.

- **FEES_REQUESTED**: User asked for pricing. Payload: `{}`. Impact: +20 lead score.

- **DEMO_REQUESTED**: User asked for a trial class. Payload: `{}`. Impact: +30 lead score.

- **COURSE_ADMISSION**: Staff marks a lead as enrolled in a specific course. Payload: `{"course": "Name", "staff": "Owner"}`. Impact: +50 lead score. Drives Admission Analytics.

- **LEAD_REASSIGNED**: Staff ownership changed. Payload: `{"from": "Old", "to": "New", "by": "Admin"}`. Significance: Accountability audit trail.

- **MANUAL_MESSAGE**: Staff sent a manual WhatsApp reply. Payload: `{"staff": "Name"}`. Impact: +5 score, clears "Needs Reply" warnings.

SECTION 6: ANALYTICS MODULES

All analytics share the same architecture: **Single Bulk Query + In-Memory Aggregation**. Time complexity: O(N). Queries are optimized to prevent proxy disconnects.

1. **Lead Analytics** (`/crm/analytics`): Business Purpose: View aggregate lead volume, temperature (Hot/Warm), and conversion funnels.

2. **Admission Analytics** (`/crm/admission-analytics`): Business Purpose: Track course-level enrollments and individual staff conversion rates. Reads `ADMISSION_OWNER` locks.

3. **Revenue Analytics** (`/crm/revenue-analytics`): Configured structurally but currently displays "Tracking Not Yet Configured". Relies on `is_admitted`.

4. **Health Dashboard** (`/crm/health`): Identifies CRM data rot (e.g., Unassigned Hot Leads, Missing Admission Data, Duplicate Staff Naming). Outputs a Health Score %.

5. **Staff Performance**: Built into Admissions and Health dashboards to calculate conversion percentages per staff.

6. **Action Center** (`/crm/action-center`): Read-only queue prioritizing today's actionable leads (Hot Leads, Pending Demos, Unassigned Leads). Deduplicates queries.

7. **Operations Command Center** (`/crm/operations`): The master counselor view combining Action Center queues and Health Dashboard warnings into one pane of glass.

SECTION 7: STAFF MANAGEMENT SYSTEM

Implementation details (Phase 9.2A-Lite):

- **Registry**: `app/data/staff_master.json`. A flat JSON dictionary mapping uppercase staff codes to `{"display_name", "role", "active"}`.

- **Activation/Deactivation**: Uses a toggle mechanism (`active: true|false`). NO hard deletes, ensuring historical assignments in the database never break.

- **Dropdown Assignment**: The CRM UI dynamically generates the `assigned_staff` `<select>` based on active JSON records. Protects against fragmentation (e.g., "kiran" vs "KIRAN").

- **Analytics Compatibility**: 100%. The dropdown submits standard strings to `ConversationState`, keeping SQLAlchemy legacy logic perfectly intact.

- **Future Login Migration Path**: In Phase 9.3, `staff_master.json` will be migrated into a `users` Postgres table to support password hashing and session tokens.

SECTION 8: ACCOUNTABILITY SYSTEM

Admission Ownership Locking (`ADMISSION_OWNER`):

When a staff member triggers a `COURSE_ADMISSION` event, their name is permanently embedded in the event's JSON payload. If the lead is later reassigned, the historical admission remains credited to the original closer in Admission Analytics.

Lead Reassignment Tracking (`LEAD_REASSIGNED`):

A strict UI hook detects `old_staff != new_staff` on form submission. It fires a `LEAD_REASSIGNED` event logging the transition, which is rendered dynamically in the Lead Journey timeline.

Message Auditing (`MESSAGE_OWNER`):

Manual replies via the CRM embed the current `assigned_staff` into a `MANUAL_MESSAGE` LeadEvent, ensuring counselors are credited for outreach.

SECTION 9: ROUTE INVENTORY

URL	Method	Purpose	Template Used	Access
`/crm/leads`	GET	Master list of all leads.	`crm_leads.html`	Read-only
`/crm/lead/<phone>`	GET	Detailed lead profile and timeline.	`crm_lead_detail.html`	Read-only
`/crm/lead/<phone>/update`	POST	Updates status, staff, and admissions.	N/A (Redirects)	Write
`/crm/lead/<phone>/send`	POST	Sends manual WhatsApp messages.	N/A (Redirects)	Write
`/crm/analytics`	GET	Global CRM funnel health.	`crm_analytics.html`	Read-only
`/crm/admission-analytics`	GET	Conversion tracking.	`crm_admission_analytics.html`	Read-only
`/crm/revenue-analytics`	GET	Revenue placeholder.	`crm_revenue_analytics.html`	Read-only
`/crm/health`	GET	Data integrity checks.	`crm_health.html`	Read-only
`/crm/action-center`	GET	Prioritized work queues.	`crm_action_center.html`	Read-only
`/crm/operations`	GET	Centralized Operations Command.	`crm_operations.html`	Read-only
`/crm/staff-management`	GET/POST	Manage JSON staff registry.	`crm_staff_management.html`	Read/Write

Note: All routes are guarded by `?key=ADMIN_KEY`.

SECTION 10: DEPLOYMENT ARCHITECTURE

- **Environment**: Deployed on Railway.

- **Database**: Railway-managed PostgreSQL. Connection utilizes `psycopg2` via SQLAlchemy.

- **Connection Pooling**: Due to Railway Proxy disconnect issues (stale connections), SQLAlchemy `engine_options` MUST include `pool_pre_ping=True` and `pool_recycle=300`. This ensures dead connections are dropped gracefully before executing queries.

- **Production Deployment Process**: Push to `main` branch triggers an automatic Railway build.

- **Rollback Process**: Railway allows one-click rollback to a previous deployment SHA. Because the CRM relies on event-sourcing (append-only), code rollbacks are perfectly safe and do not corrupt data.

SECTION 11: PRODUCTION INCIDENT HISTORY

1. **SECRET_KEY Flash Failure**

   - **Root Cause**: Attempted to use Flask `flash()` for notifications, but the app lacked a `SECRET_KEY` in environment config, causing session crashes.

   - **Resolution**: Replaced `flash()` with a query-parameter driven alert system (`?msg=` and `?err=`).

   - **Lessons Learned**: Avoid Flask sessions until formal authentication is implemented.

2. **500 Error Database Disconnect**

   - **Root Cause**: `psycopg2.OperationalError` (server closed connection unexpectedly). Triggered by Railway's TCP proxy severing idle connections, leading to `sqlalchemy.exc.InvalidatePoolError`.

   - **Resolution**: Implemented "Single Bulk Query" architecture to load all data instantly, minimizing connection time. Validated `pool_pre_ping` configuration.

   - **Lessons Learned**: Never use N+1 queries in Jinja templates (e.g., lazy loading `.events` inside a loop). Always resolve data in Python memory.

SECTION 12: SECURITY MODEL

- **Current Architecture**: "Security through Obscurity" + Shared Secret. All sensitive endpoints require `?key=ADMIN_KEY`.

- **Risks**: The key can be leaked or shared. Counselors have the same UI access level as Admins (they can edit other staff's leads).

- **Future Roadmap**: The transition to `users` table and JWT/Flask-Login sessions (Phase 9.3) will introduce true Role-Based Access Control (RBAC), mapping sessions to the `staff_master.json` identities.

SECTION 13: KNOWN TECHNICAL DEBT

1. **Missing Login System (High)**: Handled currently via a shared URL key. Needs formal session management.

2. **JSON Staff Registry (Medium)**: `staff_master.json` is a robust shim, but must eventually be migrated to Postgres for relational integrity (Foreign Keys).

3. **No Bulk Reassignment (Low)**: If a counselor leaves, their leads must be manually reassigned one-by-one.

4. **No RBAC (High)**: Staff can currently view the entire CRM. They need restricted views (only seeing their own assigned leads).

SECTION 14: FUTURE ROADMAP

Completed:

- Phases 1-8: Core CRM, Journey Tracking, Analytics, Health Dashboards.

- Phase 9.1: Accountability Audit Trails.

- Phase 9.2A-Lite: Flat-file Master Staff Management.

Next Recommended:

- **Phase 9.3 (Authentication Layer)**: Migrate `staff_master.json` to Postgres `users`. Implement Flask-Login. Introduce standard password hashing (Bcrypt).

- **Phase 9.4 (RBAC Boundaries)**: Implement Row-Level Security filters. Modify `/crm/leads` so `role="STAFF"` only sees rows where `assigned_user_id == current_user.id`.

- **Phase 10 (Revenue & Tasks)**: Formalize the revenue model (payments table) and task manager (callbacks/reminders).

SECTION 15: NEW DEVELOPER ONBOARDING GUIDE

Welcome to Oxford CRM!

- **System Overview**: This is an Event-Driven CRM. Do not alter past records; append new ones.

- **Important Files**:

  - `app/routes/admin.py`: The heart of the CRM logic.

  - `app/models.py`: Database definitions.

  - `templates/`: UI components.

- **Files NEVER to modify without review**: `models.py` (Schema changes lock production databases and risk breaking analytics).

- **Debugging Procedure**: Check Railway logs. If a 500 error occurs, it is almost always an N+1 SQLAlchemy query violating proxy timeouts. Rewrite as an in-memory aggregation.

- **Production Safety Checklist**:

  1. No schema changes.

  2. No N+1 queries.

  3. No `flash()` messages.

  4. Preserve legacy string formats.

SECTION 16: PROJECT MEMORY SNAPSHOTS (FOR AI ASSISTANTS)

> [!TIP]

> **To Future AI Context Processors:**

> - **Production State**: Stable, read-only analytics heavily optimized for in-memory processing.

> - **Verified Findings**: Railway disconnects idle Postgres connections. All complex analytics MUST use single bulk queries (`.all()`) and python aggregations.

> - **Verified Fixes**: Flash alerts replaced by URL parameters. N+1 loops replaced by O(N) Python dictionaries.

> - **Current Constraints**: Strict "No Schema Migration" rule in effect to protect historical data integrity.

> - **Pending Features**: Migration from `ADMIN_KEY` to actual user sessions (Phase 9.3) and Role-Based Access Control filtering.